CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

Token CLASS

SOURCE

An individual token — i.e. a word, punctuation symbol, whitespace, etc.

Token.__init__ METHOD

Construct a `Token` object.

NAME	DESCRIPTION
`vocab`	A storage container for lexical types. **TYPE:** `Vocab`
`doc`	The parent document. **TYPE:** `Doc`
`offset`	The index of the token within the document. **TYPE:** `int`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
token = doc[0]
assert token.text == "Give"
```

Token.__len__ METHOD

The number of unicode characters in the token, i.e. `token.text`.

NAME	DESCRIPTION
RETURNS	The number of unicode characters in the token. **TYPE:** `int`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
token = doc[0]
assert len(token) == 4
```

Token.set_extension CLASSMETHOD

Define a custom attribute on the `Token` which becomes available via `Token.._`. For details, see the documentation on custom attributes.

NAME	DESCRIPTION
`name`	Name of the attribute to set by the extension. For example, `"my_attr"` will be available as `token._.my_attr`. **TYPE:** `str`
`default`	Optional default value of the attribute if no getter or method is defined. **TYPE:** `Optional[Any]`
`method`	Set a custom method on the object, for example `token._.compare(other_token)`.

EXAMPLE

```
from spacy.tokens import Token
fruit_getter = lambda token: token.text in ("apple", "pear", "banana")
Token.set_extension("is_fruit", getter=fruit_getter)
doc = nlp("I have an apple")
assert doc[3]._.is_fruit
```

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

	TYPE: `Optional[Callable[[Token, …], Any]]`
`getter`	Getter function that takes the object and returns an attribute value. Is called when the user accesses the `._` attribute.
	TYPE: `Optional[Callable[[Token], Any]]`
`setter`	Setter function that takes the `Token` and a value, and modifies the object. Is called when the user writes to the `Token._` attribute.
	TYPE: `Optional[Callable[[Token, Any], None]]`
`force`	Force overwriting existing attribute.
	TYPE: `bool`

Token.get_extension CLASSMETHOD

Look up a previously registered extension by name. Returns a 4-tuple `(default, method, getter, setter)` if the extension is registered. Raises a `KeyError` otherwise.

NAME	DESCRIPTION
`name`	Name of the extension.
	TYPE: `str`
RETURNS	A `(default, method, getter, setter)` tuple of the extension.
	TYPE: `Tuple[Optional[Any], Optional[Callable], Optional[Callable], Optional[Callable]]`

EXAMPLE

```
from spacy.tokens import Token
Token.set_extension("is_fruit", default=False)
extension = Token.get_extension("is_fruit")
assert extension == (False, None, None, None)
```

Token.has_extension CLASSMETHOD

Check whether an extension has been registered on the `Token` class.

NAME	DESCRIPTION
`name`	Name of the extension to check.
	TYPE: `str`
RETURNS	Whether the extension has been registered.
	TYPE: `bool`

EXAMPLE

```
from spacy.tokens import Token
Token.set_extension("is_fruit", default=False)
assert Token.has_extension("is_fruit")
```

Token.remove_extension CLASSMETHOD

Remove a previously registered extension.

EXAMPLE

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

`name`	Name of the extension.
	TYPE: `str`
RETURNS	A `(default, method, getter, setter)` tuple of the removed extension.
	TYPE: `Tuple[Optional[Any], Optional[Callable], Optional[Callable], Optional[Callable]]`

```
assert not Token.has_extension("is_fruit")
```

Token.check_flag METHOD

Check the value of a boolean flag.

NAME	DESCRIPTION
`flag_id`	The attribute ID of the flag to check.
	TYPE: `int`
RETURNS	Whether the flag is set.
	TYPE: `bool`

EXAMPLE

```
from spacy.attrs import IS_TITLE
doc = nlp("Give it back! He pleaded.")
token = doc[0]
assert token.check_flag(IS_TITLE) == True
```

Token.similarity METHOD NEEDS MODEL

Compute a semantic similarity estimate. Defaults to cosine over vectors.

NAME	DESCRIPTION
other	The object to compare with. By default, accepts `Doc`, `Span`, `Token` and `Lexeme` objects.
	TYPE: `Union[Doc,Span,Token,Lexeme]`
RETURNS	A scalar similarity score. Higher is more similar.
	TYPE: `float`

EXAMPLE

```
apples, _, oranges = nlp("apples and oranges")
apples_oranges = apples.similarity(oranges)
oranges_apples = oranges.similarity(apples)
assert apples_oranges == oranges_apples
```

Token.nbor METHOD

Get a neighboring token.

NAME	DESCRIPTION
`i`	The relative position of the token to get. Defaults to `1`.
	TYPE: `int`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
give_nbor = doc[0].nbor()
assert give_nbor.text == "it"
```

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

RETURNS	The token at position `self.doc[self.i+i]`. **TYPE:** Token

Token.set_morph METHOD

Set the morphological analysis from a UD FEATS string, hash value of a UD FEATS string, features dict or `MorphAnalysis`. The value `None` can be used to reset the morph to an unset state.

NAME	DESCRIPTION
features	The morphological features to set. **TYPE:** `Union[int, dict, str,MorphAnalysis, None]`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
doc[0].set_morph("Mood=Imp|VerbForm=Fin")
assert "Mood=Imp" in doc[0].morph
assert doc[0].morph.get("Mood") == ["Imp"]
```

Token.has_morph METHOD

Check whether the token has annotated morph information. Return `False` when the morph annotation is unset/missing.

NAME	DESCRIPTION
RETURNS	Whether the morph annotation is set. **TYPE:** `bool`

Token.is_ancestor METHOD NEEDS MODEL

Check whether this token is a parent, grandparent, etc. of another in the dependency tree.

NAME	DESCRIPTION
descendant	Another token. **TYPE:** Token
RETURNS	Whether this token is the ancestor of the descendant. **TYPE:** `bool`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
give = doc[0]
it = doc[1]
assert give.is_ancestor(it)
```

Token.ancestors PROPERTY NEEDS MODEL

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

NAME	DESCRIPTION
YIELDS	A sequence of ancestor tokens such that `ancestor.is_ancestor(self)`. **TYPE:** `Token`

```
doc = nlp("Give it back! He pleaded.")
it_ancestors = doc[1].ancestors
assert [t.text for t in it_ancestors] == ["Give"]
he_ancestors = doc[4].ancestors
assert [t.text for t in he_ancestors] == ["pleaded"]
```

Token.conjuncts PROPERTY NEEDS MODEL

A tuple of coordinated tokens, not including the token itself.

NAME	DESCRIPTION
RETURNS	The coordinated tokens. **TYPE:** `Tuple[Token, …]`

EXAMPLE

```
doc = nlp("I like apples and oranges")
apples_conjuncts = doc[2].conjuncts
assert [t.text for t in apples_conjuncts] == ["oranges"]
```

Token.children PROPERTY NEEDS MODEL

A sequence of the token's immediate syntactic children.

NAME	DESCRIPTION
YIELDS	A child token such that `child.head == self`. **TYPE:** `Token`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
give_children = doc[0].children
assert [t.text for t in give_children] == ["it", "back", "!"]
```

Token.lefts PROPERTY NEEDS MODEL

The leftward immediate children of the word in the syntactic dependency parse.

NAME	DESCRIPTION
YIELDS	A left-child of the token. **TYPE:** `Token`

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
lefts = [t.text for t in doc[3].lefts]
assert lefts == ["New"]
```

Token.rights PROPERTY NEEDS MODEL

The rightward immediate children of the word in the syntactic dependency parse.

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

YIELDS	A right-child of the token. **TYPE:** `Token`

```
rights = [t.text for t in doc[3].rights]
assert rights == ["in"]
```

Token.n_lefts PROPERTY NEEDS MODEL

The number of leftward immediate children of the word in the syntactic dependency parse.

NAME	DESCRIPTION
RETURNS	The number of left-child tokens. **TYPE:** `int`

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
assert doc[3].n_lefts == 1
```

Token.n_rights PROPERTY NEEDS MODEL

The number of rightward immediate children of the word in the syntactic dependency parse.

NAME	DESCRIPTION
RETURNS	The number of right-child tokens. **TYPE:** `int`

EXAMPLE

```
doc = nlp("I like New York in Autumn.")
assert doc[3].n_rights == 1
```

Token.subtree PROPERTY NEEDS MODEL

A sequence containing the token and all the token's syntactic descendants.

NAME	DESCRIPTION
YIELDS	A descendant token such that `self.is_ancestor(token)` or `token == self`. **TYPE:** `Token`

EXAMPLE

```
doc = nlp("Give it back! He pleaded.")
give_subtree = doc[0].subtree
assert [t.text for t in give_subtree] == ["Give", "it", "back", "!"]
```

Token.has_vector PROPERTY NEEDS MODEL

A boolean value indicating whether a word vector is associated with the token.

EXAMPLE

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

RETURNS	Whether the token has a vector data attached. **TYPE:** `bool`

Token.vector PROPERTY NEEDS MODEL

A real-valued meaning representation.

NAME	DESCRIPTION
RETURNS	A 1-dimensional array representing the token's vector. **TYPE:** `numpy.ndarray[ndim=1, dtype=float32]`

EXAMPLE

```
doc = nlp("I like apples")
apples = doc[2]
assert apples.vector.dtype == "float32"
assert apples.vector.shape == (300,)
```

Token.vector_norm PROPERTY NEEDS MODEL

The L2 norm of the token's vector representation.

NAME	DESCRIPTION
RETURNS	The L2 norm of the vector representation. **TYPE:** `float`

EXAMPLE

```
doc = nlp("I like apples and pasta")
apples = doc[2]
pasta = doc[4]
apples.vector_norm  # 6.89589786529541
pasta.vector_norm  # 7.759851932525635
assert apples.vector_norm != pasta.vector_norm
```

Attributes

NAME	DESCRIPTION
`doc`	The parent document. **TYPE:** `Doc`
`lex` V3.0	The underlying lexeme. **TYPE:** `Lexeme`
`sent`	The sentence span that this token is a part of. **TYPE:** `Span`
`text`	Verbatim text content. **TYPE:** `str`
`text_with_ws`	Text content, with trailing space character if present. **TYPE:** `str`
`whitespace_`	Trailing space character if present.

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

	TYPE: str
`orth`	ID of the verbatim text content.
	TYPE: int
`orth_`	Verbatim text content (identical to `Token.text`). Exists mostly for consistency with the other attributes.
	TYPE: str
`vocab`	The vocab object of the parent `Doc`.
	TYPE: vocab
`tensor`	The token's slice of the parent `Doc`'s tensor.
	TYPE: numpy.ndarray
`head`	The syntactic parent, or "governor", of this token.
	TYPE: Token
`left_edge`	The leftmost token of this token's syntactic descendants.
	TYPE: Token
`right_edge`	The rightmost token of this token's syntactic descendants.
	TYPE: Token
`i`	The index of the token within the parent document.
	TYPE: int
`ent_type`	Named entity type.
	TYPE: int
`ent_type_`	Named entity type.
	TYPE: str
`ent_iob`	IOB code of named entity tag. `3` means the token begins an entity, `2` means it is outside an entity, `1` means it is inside an entity, and `0` means no entity tag is set.
	TYPE: int
`ent_iob_`	IOB code of named entity tag. "B" means the token begins an entity, "I" means it is inside an entity, "O" means it is outside an entity, and "" means no entity tag is set.
	TYPE: str
`ent_kb_id`	Knowledge base ID that refers to the named entity this token is a part of, if any.
	TYPE: int
`ent_kb_id_`	Knowledge base ID that refers to the named entity this token is a part of, if any.
	TYPE: str
`ent_id`	ID of the entity the token is an instance of, if any. Currently not used, but potentially for coreference resolution.
	TYPE: int
`ent_id_`	ID of the entity the token is an instance of, if any. Currently not used, but potentially for coreference resolution.
	TYPE: str
`lemma`	Base form of the token, with no inflectional suffixes.
	TYPE: int
`lemma_`	Base form of the token, with no inflectional suffixes.
	TYPE: str
`norm`	The token's norm, i.e. a normalized form of the token text. Can be set in the language's tokenizer exceptions.
	TYPE: int
`norm_`	The token's norm, i.e. a normalized form of the token text. Can be set in the language's tokenizer exceptions.
	TYPE: str

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

`lower`	Lowercase form of the token. **TYPE:** `int`
`lower_`	Lowercase form of the token text. Equivalent to `Token.text.lower()` . **TYPE:** `str`
`shape`	Transform of the token's string to show orthographic features. Alphabetic characters are replaced by `x` or `X` , and numeric characters are replaced by `d` , and sequences of the same character are truncated after length 4. For example, `"Xxxx"` or `"dd"` . **TYPE:** `int`
`shape_`	Transform of the token's string to show orthographic features. Alphabetic characters are replaced by `x` or `X` , and numeric characters are replaced by `d` , and sequences of the same character are truncated after length 4. For example, `"Xxxx"` or `"dd"` . **TYPE:** `str`
`prefix`	Hash value of a length-N substring from the start of the token. Defaults to `N=1` . **TYPE:** `int`
`prefix_`	A length-N substring from the start of the token. Defaults to `N=1` . **TYPE:** `str`
`suffix`	Hash value of a length-N substring from the end of the token. Defaults to `N=3` . **TYPE:** `int`
`suffix_`	Length-N substring from the end of the token. Defaults to `N=3` . **TYPE:** `str`
`is_alpha`	Does the token consist of alphabetic characters? Equivalent to `token.text.isalpha()` . **TYPE:** `bool`
`is_ascii`	Does the token consist of ASCII characters? Equivalent to `all(ord(c) < 128 for c in token.text)` . **TYPE:** `bool`
`is_digit`	Does the token consist of digits? Equivalent to `token.text.isdigit()` . **TYPE:** `bool`
`is_lower`	Is the token in lowercase? Equivalent to `token.text.islower()` . **TYPE:** `bool`
`is_upper`	Is the token in uppercase? Equivalent to `token.text.isupper()` . **TYPE:** `bool`
`is_title`	Is the token in titlecase? Equivalent to `token.text.istitle()` . **TYPE:** `bool`
`is_punct`	Is the token punctuation? **TYPE:** `bool`
`is_left_punct`	Is the token a left punctuation mark, e.g. `"("` ? **TYPE:** `bool`
`is_right_punct`	Is the token a right punctuation mark, e.g. `")"` ? **TYPE:** `bool`
`is_sent_start`	Does the token start a sentence? `bool` or `None` if unknown. Defaults to `True` for the first token in the `Doc` .
`is_sent_end`	Does the token end a sentence? `bool` or `None` if unknown.
`is_space`	Does the token consist of whitespace characters? Equivalent to `token.text.isspace()` . **TYPE:** `bool`
`is_bracket`	Is the token a bracket? **TYPE:** `bool`

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

`is_quote`	Is the token a quotation mark? **TYPE:** `bool`
`is_currency`	Is the token a currency symbol? **TYPE:** `bool`
`like_url`	Does the token resemble a URL? **TYPE:** `bool`
`like_num`	Does the token represent a number? e.g. “10.9”, “10”, “ten”, etc. **TYPE:** `bool`
`like_email`	Does the token resemble an email address? **TYPE:** `bool`
`is_oov`	Is the token out-of-vocabulary (i.e. does it not have a word vector)? **TYPE:** `bool`
`is_stop`	Is the token part of a “stop list”? **TYPE:** `bool`
`pos`	Coarse-grained part-of-speech from the Universal POS tag set. **TYPE:** `int`
`pos_`	Coarse-grained part-of-speech from the Universal POS tag set. **TYPE:** `str`
`tag`	Fine-grained part-of-speech. **TYPE:** `int`
`tag_`	Fine-grained part-of-speech. **TYPE:** `str`
`morph` V3.0	Morphological analysis. **TYPE:** `MorphAnalysis`
`dep`	Syntactic dependency relation. **TYPE:** `int`
`dep_`	Syntactic dependency relation. **TYPE:** `str`
`lang`	Language of the parent document’s vocabulary. **TYPE:** `int`
`lang_`	Language of the parent document’s vocabulary. **TYPE:** `str`
`prob`	Smoothed log probability estimate of token’s word type (context-independent entry in the vocabulary). **TYPE:** `float`
`idx`	The character offset of the token within the parent document. **TYPE:** `int`
`sentiment`	A scalar value indicating the positivity or negativity of the token. **TYPE:** `float`
`lex_id`	Sequential ID of the token’s lexical type, used to index into tables, e.g. for word vectors. **TYPE:** `int`
`rank`	Sequential ID of the token’s lexical type, used to index into tables, e.g. for word vectors. **TYPE:** `int`

CONTAINERS
Doc
DocBin
Example
Language
Lexeme
Span
SpanGroup
Token

PIPELINE
AttributeRuler
CoreferenceResolver
DependencyParser
EditTreeLemmatizer
EntityLinker
EntityRecognizer
EntityRuler
Large Language Models
Lemmatizer
Morphologizer
SentenceRecognizer
Sentencizer
SpanCategorizer
SpanFinder
SpanResolver
SpanRuler
Tagger
TextCategorizer
Tok2Vec
Tokenizer
TrainablePipe
Transformer
Other Functions

MATCHERS
DependencyMatcher
Matcher
PhraseMatcher

OTHER
Attributes
Corpus
InMemoryLookupKB
KnowledgeBase
Lookups
MorphAnalysis
Morphology
Scorer
StringStore
Vectors
Vocab

CYTHON
Architecture
Classes

`cluster`	Brown cluster ID. **TYPE:** `int`
`_`	User space for adding custom attribute extensions. **TYPE:** `Underscore`

SUGGEST EDITS

© 2016-2023 Explosion